Fényképalbum alkalmazás

Skálázhatóság dokumentáció

Készítette:

Várkonyi Gábor, E14KQR

Automatikus skálázódás konfigurációja (Elastic Beanstalk

Architektúra áttekintése

A fényképalbum alkalmazás AWS Elastic Beanstalk környezetben fut, load balancerrel ellátott architektúrában. A rendszer több komponensből áll:

- Flask alapú webalkalmazás (Gunicorn + Nginx)
- Amazon RDS (PostgreSQL adatbázis)
- Amazon S3 (képfájlok tárolása)
- Elastic Load Balancer
- Auto Scaling Group

Az alkalmazás horizontálisan skálázható, mivel:

- a képfájlok S3-ban találhatók,
- az adatok RDS-ben vannak,
- az alkalmazásszerverek állapotmentesek.

Skálázási konfiguráció

Az Elastic Beanstalk környezetben az alábbi beállításokat alkalmaztam:

- Environment type: **Load balanced**
- Minimum instance szám: **1**
- Maximum instance szám: **4**
- Instance típus: **t3.micro**

Skálázási szabály

A skálázás CPU-alapú triggerrel történt:

- Metric: **CPUUtilization**
- Scale out:
  - Threshold: 40–50%
  - Duration: 5 perc
- Scale in:
  - Threshold: 20%
  - Duration: 10 perc
- Scaling increment:
  - +1 / -1 instance

A CPU-alapú skálázás azért megfelelő, mert az alkalmazás terhelése főként:

- HTTP request feldolgozásból,
- adatbázis műveletekből,
- képfeltöltésből

származik, nem pedig hálózati forgalomból

Terheléspróba jegyzőkönyve

Használt eszköz

A terheléses teszthez a **Locust** nevű eszközt használtam.

A teszt szimulálta:

- regisztráció
- bejelentkezés
- képek listázása
- kép megnyitása
- kép feltöltése
- kép törlése

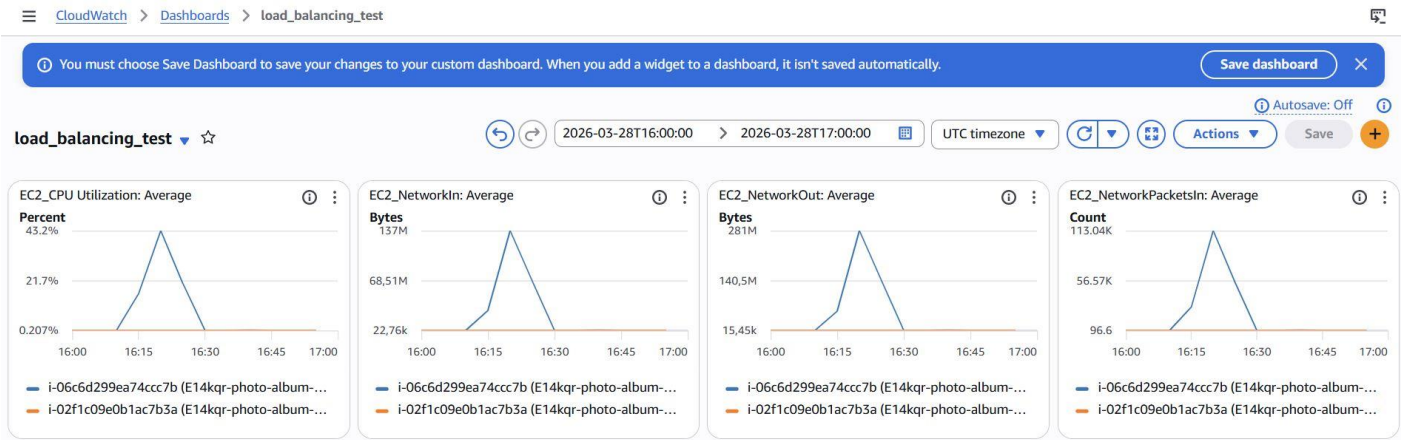
Locust konfiguráció és kód

A kód megtalálható a dokumentum végén.

- Users: **50**
- Spawn rate: **5 user/sec**
- Run time: ~10 perc
- Host: Elastic Beanstalk URL

Terhelés hatása (eredmények)

1. CPU terhelés



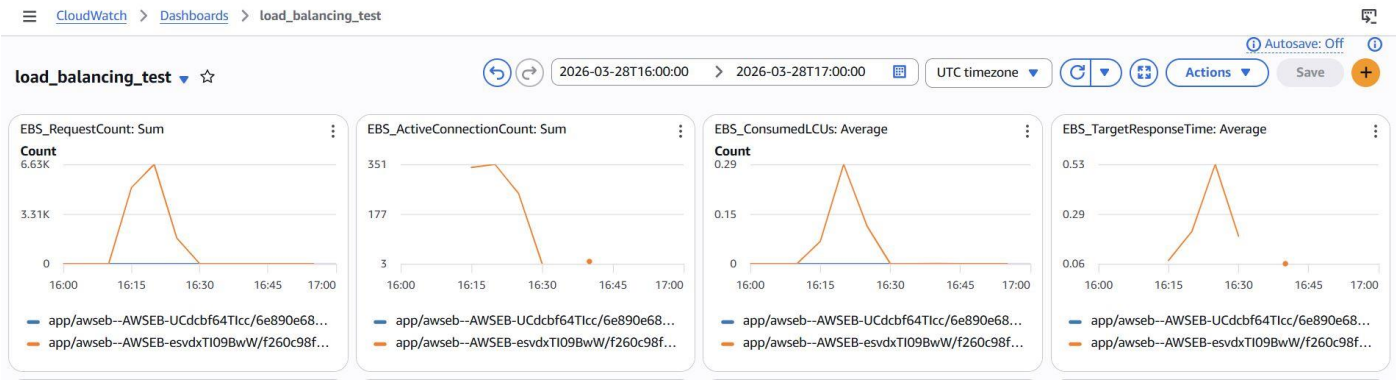
Megfigyelések:

- CPU ~0% → ~43%-ig nőtt
- egyértelmű terhelésnövekedés történt

-> Következtetés:

- a Locust sikeresen terhelte az alkalmazást

2. Load balancer terhelés



- ~6000+ request
- aktív kapcsolatok nőnek
- response time is emelkedik

3. Auto Scaling működés

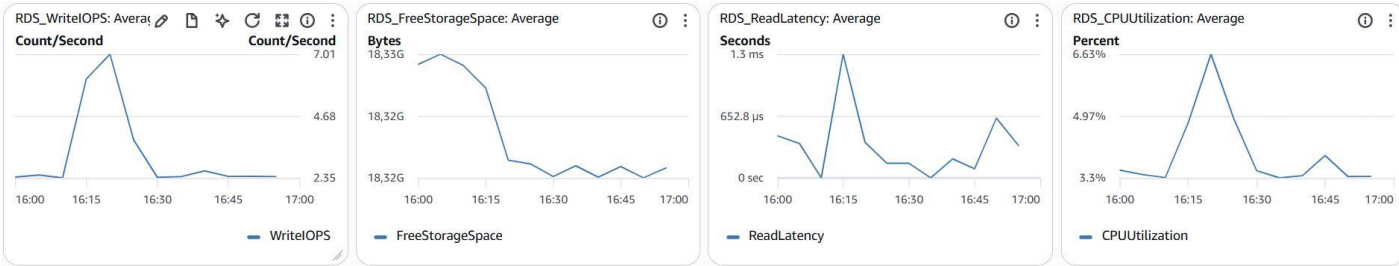
Name	Launch template/configuration	Instances	Status	Desired capacity	Min	Max
awseb-e-fgkfgpm8ng-stack-AWSEBAutoScalingGroup-1NnEGBQustOz	AWSEBEC2LaunchTemplate_1c7OyydmU	2	-	2	1	4

- Instances: **2**
- Desired capacity: **2**
- Min: 1 → Max: 4

->Itt látható hogy:

- kezdetben 1 instance
- terhelés hatására → 2 instance
- tehát megtörtént a scale out

4. RDS terhelés



- WriteIOPS nő
- CPU ~6%-ig nő

-> Következtetés:

- adatbázis is terhelődött
- de nem bottleneck

Automatikus felskálázás bizonyítéka

A CloudWatch és az Auto Scaling Group adatai alapján:

- a CPU terhelés növekedése után

- az Auto Scaling Group növelte a példányszámot
- a rendszer 1 → 2 instance-ra skálázódott

Ez igazolja az automatikus felskálázás működését.

Tanulságok

A CPU-alapú skálázás működik

A teszt igazolta, hogy:

- a CPU megfelelő indikátora a terhelésnek
- a rendszer automatikusan reagál

-> fontos tanulság:

- a NetworkOut nem lett volna megfelelő metrika

Az architektúra jól skálázható

Az alkalmazás:

- nem tárol fájlokat lokálisan
- S3 + RDS használat

-> ezért:

- új instance indítása egyszerű
- nincs state probléma

A skálázás nem azonnali

Megfigyelhető volt, hogy:

- a skálázás késleltetve történik
- több perc kell a triggerhez

-> tanulság:

- a threshold és cooldown kritikus

A terheléses teszt realisztikus volt

A Locust:

- többféle endpointot hívott
- nem csak GET requestek voltak
- upload is történt

-> emiatt:

- valósabb terhelést kaptunk

Limitációk

- t3.micro instance gyorsan telítődik
- csak 2 instance-ig skálázott (nem érte el a max-ot)
- hosszabb teszttel több scale step is látható lenne

Locust kód:

```
from locust import HttpUser, task, between
import uuid
import random
import re

class PhotoAlbumUser(HttpUser):
    wait_time = between(1, 3)

    def on_start(self):
        self.email = f"locust_{uuid.uuid4().hex[:12]}@example.com"
        self.password = "Teszt123!"
        self.owned_photo_ids = []

        self.register()
        self.login()

    def register(self):
        with self.client.post(
            "/register",
            data={
                "email": self.email,
                "password": self.password,
            },
            name="POST /register",
            allow_redirects=True,
            catch_response=True,
        ) as response:
            if response.status_code == 200:
                response.success()
            else:
                response.failure(f"Regisztráció sikertelen: {response.status_code}")

    def login(self):
        with self.client.post(
            "/login",
            data={
                "email": self.email,
                "password": self.password,
            },
            name="POST /login",
            allow_redirects=True,
            catch_response=True,
        ) as response:
```

```
        if response.status_code == 200:
            response.success()
        else:
            response.failure(f"Login sikertelen: {response.status_code}")

    @task(4)
    def browse_index(self):
        sort = random.choice(["date", "name"])
        direction = random.choice(["asc", "desc"])

        with self.client.get(
            f"/?sort={sort}&dir={direction}",
            name="GET /?sort&dir",
            catch_response=True,
        ) as response:
        if response.status_code == 200:
            response.success()
        else:
            response.failure(f"Főoldal lekérés sikertelen: {response.status_code}")

    @task(2)
    def upload_photo(self):
        photo_name = f"Locust kép {uuid.uuid4().hex[:8]}"
        filename = f"{uuid.uuid4().hex}.jpg"

        fake_jpeg = (
            b"\xff\xd8\xff\xe0"
            + b"JFIF\x00"
            + bytes(random.getrandbits(8) for _ in range(4096))
            + b"\xff\xd9"
        )

        files = {
            "file": (filename, fake_jpeg, "image/jpeg")
        }

        data = {
            "name": photo_name
        }

        with self.client.post(
            "/upload",
            data=data,
            files=files,
            name="POST /upload",
            allow_redirects=True,
            catch_response=True,
```

```
    ) as response:
        if response.status_code == 200:
            photo_id = self._extract_latest_photo_id(response.text)
            if photo_id:
                self.owned_photo_ids.append(photo_id)
            response.success()
        else:
            response.failure(f"Feltöltés sikertelen: {response.status_code}")

@task(3)
def view_photo(self):
    photo_id = None

    if self.owned_photo_ids and random.random() < 0.7:
        photo_id = random.choice(self.owned_photo_ids)
    else:
        photo_id = random.randint(1, 30)

    with self.client.get(
        f"/photo/{photo_id}",
        name="GET /photo/[id]",
        catch_response=True,
    ) as response:
        if response.status_code in (200, 404):
            response.success()
        else:
            response.failure(f"Fotó megnyitás sikertelen: {response.status_code}")

@task(1)
def delete_own_photo(self):
    if not self.owned_photo_ids:
        return

    photo_id = random.choice(self.owned_photo_ids)

    with self.client.post(
        f"/delete/{photo_id}",
        name="POST /delete/[id]",
        allow_redirects=True,
        catch_response=True,
    ) as response:
        if response.status_code == 200:
            try:
                self.owned_photo_ids.remove(photo_id)
            except ValueError:
                pass
            response.success()
```

```
        else:
            response.failure(f"Törlés sikertelen: {response.status_code}")

def _extract_latest_photo_id(self, html: str):
    matches = re.findall(r'/photo/(\\d+)', html)
    if matches:
        try:
            return int(matches[-1])
        except ValueError:
            return None
    return None
```